Skinned Mesh Separator

User Guide

Chris Burns

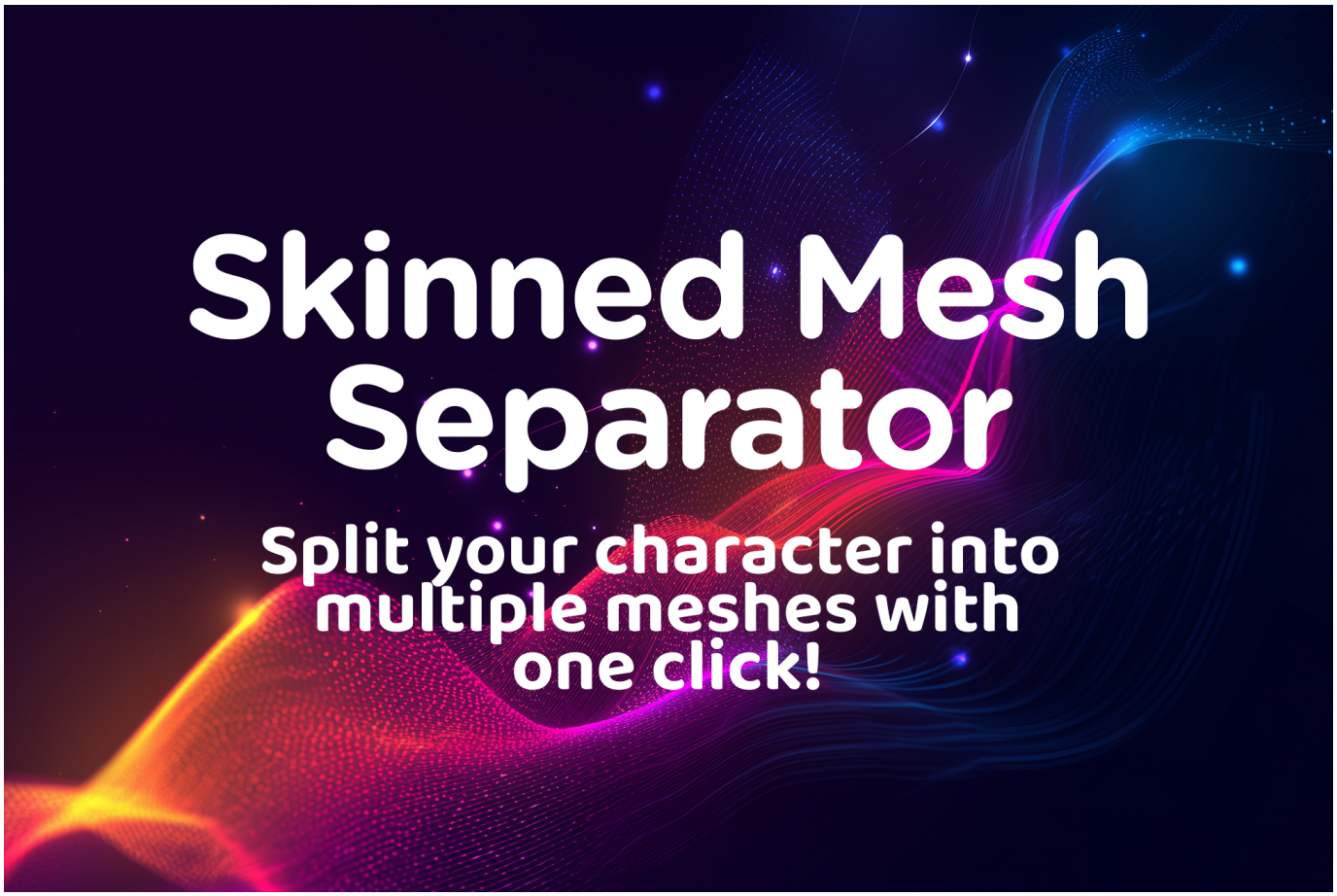
Copyright Chris Burns

Table of Contents

1. Skinned Mesh Separator 3
   - 1.1 Use Cases 4
   - 1.2 Unity Version Compatibility 4
   - 1.3 Support 5
2. Getting Started 6
   - 2.1 Installation 6
   - 2.2 Quick Start 7
3. Usage 9
   - 3.1 Tool Overview 9
   - 3.2 Extracting Meshes 12
   - 3.3 Common Workflows 14
4. Reference 16
   - 4.1 All Features 16
   - 4.2 Limitations 18
5. Support 20
   - 5.1 Discord 20
   - 5.2 Email 20
   - 5.3 Asset Store 20
   - 5.4 Leaving a Review 20
   - 5.5 Changelog 20
   - 5.6 Other Assets by Chris Burns 21

Copyright Chris Burns

1. Skinned Mesh Separator



Split skinned character meshes by bone — directly in Unity. No external DCC tools required.

Skinned Mesh Separator is a Unity Editor window tool that extracts mesh sections from any `SkinnedMeshRenderer` based on bone influence. Select a bone, click extract, and get a clean new mesh asset saved directly to your project.

Copyright Chris Burns

Your original character is never modified.

Fast Setup

Install via Package Manager and open from `Tools → Skinned Mesh Separator`. No configuration required.

→ Installation

Auto-Find Bones

Automatically detect head and shoulder bones from your rig's bone list with one click.

→ Tool Overview

Any Rig Type

Works with humanoid, generic, creature, and custom bone hierarchies — any SkinnedMeshRenderer.

→ Common Workflows

Non-Destructive

Extracted meshes save as new assets. The source mesh and character are never touched.

→ Extracting Meshes

1.1 Use Cases

Workflow	Description
Modular Characters	Isolate body regions to show/hide under armour or clothing layers
First-Person Games	Generate a headless or armless body mesh aligned to a camera rig
LOD Creation	Build simplified upper or lower body meshes for performance budgets
Gameplay Effects	Pre-separate body parts for damage, dismemberment, or prosthetics
Runtime Mesh Switching	Use extracted meshes as swappable `SkinnedMeshRenderer` components

1.2 Unity Version Compatibility

Unity Version	Built-in RP	URP	HDRP
2020.3 LTS and above	☑	☑	☑
2021.x	☑	☑	☑
2022.x	☑	☑	☑
2023.x	☑	☑	☑
Unity 6.x	☑	☑	☑

Copyright Chris Burns

1.3 Support

- **Discord:** discord.gg/DSUd2QcyHZ
- **Email:** info@chrisburns.com.au
- **Asset Store:** Leave a rating

Enjoying the tool?

If Skinned Mesh Separator saved you time, a quick star rating on the Asset Store helps other developers find it — and helps fund future updates. Thank you!

Copyright Chris Burns

2. Getting Started

2.1 Installation

2.1.1 Requirements

- Unity **2020.3 LTS or above** (tested through Unity 6.x)
- Any render pipeline (Built-in, URP, HDRP)

2.1.2 Option 1: Unity Package Manager (Recommended)

Install directly through the Unity Package Manager from the Asset Store.

1. Open Unity and navigate to **Window** → **Package Manager**
2. Select **My Assets** from the dropdown
3. Find **Skinned Mesh Separator** and click **Download**, then **Import**

2.1.3 Option 2: Manual Installation

If you have the `.cs` file directly:

1. In your Unity project, create an `Editor` folder inside `Assets` if one doesn't exist
2. Place `SkinnedMeshSeparator.cs` inside the `Editor` folder

```
Assets/
└── SkinnedMeshSeparator/
    └── Editor/
        └── SkinnedMeshSeparator.cs
```

> ⚠ **Editor folder required**
>
> The script **must** be placed inside a folder named `Editor`. Unity will throw compile errors if it is placed in a non-editor folder, because the script references `UnityEditor` APIs that are not available in builds.

2.1.4 Verify Installation

Once installed, the tool will appear in Unity's menu bar:

Tools → Skinned Mesh Separator

Click it to open the editor window. If you don't see the menu item, check the Console for compile errors and ensure the script is inside an `Editor` folder.

Copyright Chris Burns

2.2 Quick Start

Get your first extracted mesh in under a minute.

2.2.1 Step 1 — Open the Tool

Navigate to **Tools** → **Skinned Mesh Separator** in Unity's menu bar.

2.2.2 Step 2 — Assign a SkinnedMeshRenderer

Drag a character's `SkinnedMeshRenderer` component from the Hierarchy or Inspector into the **Skinned Mesh Renderer** field.

2.2.3 Step 3 — Find Bones

Click 🔍 **Auto-Find Bones** to automatically detect the head and shoulder bones from your rig.

> **Tip**
>
> Auto-Find works with most standard humanoid naming conventions (e.g. `Head`, `Shoulder_L`, `UpperArm_R`). If your rig uses non-standard names, assign the bones manually by dragging them from the Hierarchy.

2.2.4 Step 4 — Enable Read/Write (if needed)

If the mesh is not yet readable, click 🔧 **Enable Read/Write on Source Mesh**. This updates the model's import settings automatically — no need to dig through the Inspector.

> **Note**
>
> The source mesh must have **Read/Write Enabled** before any extraction can proceed. The tool will log an error in the Console if this step is skipped.

2.2.5 Step 5 — Extract

Click one of the preset extraction buttons:

Button	Result
Keep ONLY Head	Extracts just the head region
Remove Head	Keeps the body and arms, removes the head
Keep ONLY Arms	Extracts both arms
Remove Arms	Keeps the body and head, removes arms
Keep ONLY Other Part	Extracts the bone set in the Other Bone field
Remove Other Part	Removes the bone in the Other Bone field, keeps the rest

Copyright Chris Burns

2.2.6 Step 6 — Save

A save dialog will appear. Choose a location inside your `Assets` folder and give the mesh a name. The new `.asset` file will be created there.

Your original character mesh is unchanged.

2.2.7 Next Steps

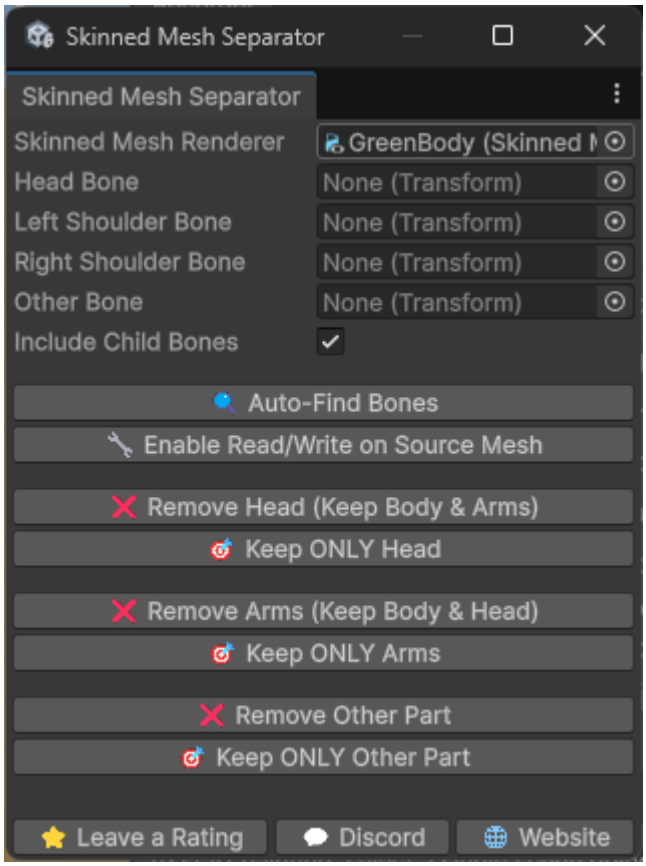
- Tool Overview — full breakdown of every field and button
- Common Workflows — real-world use case examples

Copyright Chris Burns

3. Usage

3.1 Tool Overview

Open the tool via **Tools** → **Skinned Mesh Separator**.



3.1.1 Input Fields

Skinned Mesh Renderer

Assign the `SkinnedMeshRenderer` component from the character you want to process. This is the only required field.

Head Bone

The bone that drives the head region of your character (e.g. `Head`, `Bip01_Head`). Used by the **Head** preset buttons.

Left Shoulder Bone

The bone that drives the left arm (e.g. `Shoulder_L`, `UpperArm_L`). Used by the **Arms** preset buttons.

Right Shoulder Bone

The bone that drives the right arm (e.g. `Shoulder_R`, `UpperArm_R`). Used by the **Arms** preset buttons.

Copyright Chris Burns

Other Bone

An optional field for any additional bone region — a leg, hand, tail, accessory, or any custom part. Used by the **Other Part** preset buttons.

Include Child Bones

When enabled, any extraction also includes all child bones of the selected bone. This ensures that sub-bones (e.g. jaw, fingers, eyebrows) are captured correctly.

> **Example**
>
> If your head bone has a `Jaw` child bone and `Include Child Bones` is on, the jaw geometry will be included when extracting the head.

3.1.2 Buttons

🔍 Auto-Find Bones

Scans the rig's bone list and attempts to automatically assign the Head, Left Shoulder, and Right Shoulder fields based on common naming conventions. Supports variations like:

- `head`, `Head`, `Bip01_Head`
- `leftshoulder`, `Shoulder_L`, `LeftArm`, `UpperArm_L`
- `rightshoulder`, `Shoulder_R`, `RightArm`, `UpperArm_R`

The Console will log how many bones were found (e.g. `Auto-Find Bones: found 3/3`).

🔧 Enable Read/Write on Source Mesh

Enables the **Read/Write** flag on the source mesh's import settings in one click. This is required before any extraction. The button triggers a reimport of the asset automatically.

3.1.3 Preset Extraction Buttons

All buttons require the **Skinned Mesh Renderer** to be assigned and the mesh to be readable.

Button	Bones Required	Output
Keep ONLY Head	Head Bone	Head geometry only
Remove Head (Keep Body & Arms)	Head Bone	Full mesh minus head
Keep ONLY Arms	Left + Right Shoulder	Arms geometry only
Remove Arms (Keep Body & Head)	Left + Right Shoulder	Full mesh minus arms
Keep ONLY Other Part	Other Bone	Custom region only
Remove Other Part	Other Bone	Full mesh minus custom region

Copyright Chris Burns

3.1.4 Footer Buttons

These appear at the bottom of the tool window:

Button	Action
Leave a Rating	Opens the Asset Store review page
Discord	Opens the support Discord server
Website	Opens chrisburns.com.au

Copyright Chris Burns

3.2 Extracting Meshes

This page covers how mesh extraction works under the hood and what to expect from the output.

3.2.1 How Extraction Works

The tool reads the `BoneWeight` data on each vertex of your mesh. Every vertex in a `SkinnedMeshRenderer` stores up to four bone influences (indices and weights). When you select a bone and click extract:

1. The tool collects the bone index for your selected bone (and optionally its children)
2. It scans every vertex and marks those influenced by any of those indices
3. It then iterates over every triangle — if any vertex in a triangle is influenced by the selected bone, the whole triangle is kept or removed (depending on whether you're keeping or removing that region)
4. A new `Mesh` asset is created from the filtered triangle data and saved to disk

The source mesh is **never modified**. The new mesh is a copy with a subset of the original triangles.

3.2.2 What the Output Looks Like

The extracted mesh:

- Retains all original vertex data (positions, UVs, normals, tangents, blend shapes, bone weights)
- Retains the same submesh structure as the source
- Is saved as a `.asset` file in your project

> **Open edges**
>
> Because the tool simply removes triangles, the cut boundary will have open (non-watertight) edges. This is expected — see Limitations for details on when this matters and when it doesn't.

3.2.3 Assigning the Extracted Mesh

After saving, assign the new mesh to a `SkinnedMeshRenderer` on your character:

1. Select a `SkinnedMeshRenderer` component in the Inspector
2. In the **Mesh** field, assign your new `.asset` file
3. Make sure the `SkinnedMeshRenderer`'s **Bones** array still points to the original rig — the new mesh shares the same bone indices

> **Tip**
>
> Because the extracted mesh shares bone indices with the original, it will animate correctly when bound to the same skeleton.

Copyright Chris Burns

3.2.4 Child Bone Inclusion

The **Include Child Bones** toggle controls whether child bones of the selected bone are included in the extraction query.

Include Child Bones ON (default) Include Child Bones OFF

Selecting the Head bone also captures geometry influenced by:

- Jaw / chin bones
- Eye bones
- Eyebrow bones
- Ear bones
- Any accessory bones parented to the head

Only vertices directly influenced by the selected bone are extracted. Useful when you want a precise, minimal extraction without pulling in adjacent geometry.

3.2.5 Read/Write Requirement

Unity's `Mesh` API requires a mesh to have **Read/Write Enabled** before vertex and triangle data can be accessed at edit time. If your mesh doesn't have this flag set:

- Use the 🔧 **Enable Read/Write on Source Mesh** button in the tool
- Or enable it manually: select the FBX/model asset → Inspector → **Model** tab → check **Read/Write Enabled** → Apply

Copyright Chris Burns

3.3 Common Workflows

Practical step-by-step guides for the most common use cases.

3.3.1 First-Person Body Setup

Generate a headless body mesh for a first-person game where the camera would clip through the character's head.

1. Open **Tools** → **Skinned Mesh Separator**
2. Assign the full-body `SkinnedMeshRenderer`
3. Click 🔍 **Auto-Find Bones** (or assign the Head bone manually)
4. Ensure **Include Child Bones** is enabled (to capture jaw, eyes, etc.)
5. Click **Remove Head (Keep Body & Arms)**
6. Save the mesh as `Character_BodyNoHead`
7. Assign `Character_BodyNoHead` to the `SkinnedMeshRenderer` used in your first-person camera setup

The body will animate correctly on the original skeleton without rendering a head in the player's view.

3.3.2 Modular Character System

Create separate mesh assets per body region for a character customisation system where players can swap body parts or hide regions under clothing.

1. Run the tool multiple times on the same source `SkinnedMeshRenderer`:
   - **Keep ONLY Head** → `Character_Head`
   - **Remove Head** → `Character_Body`
   - **Keep ONLY Arms** → `Character_Arms`
2. Assign each mesh to its own `SkinnedMeshRenderer`, all sharing the same skeleton/rig
3. Toggle `SkinnedMeshRenderer.enabled` at runtime to show/hide each region

> **Tip**
>
> This approach avoids duplicate animated characters — all `SkinnedMeshRenderers` can share the same `Animator` and bone hierarchy.

3.3.3 Extracting a Custom Body Region

Use the **Other Bone** field to extract any part of the mesh — a leg, a tail, a backpack, or any accessory.

1. In the **Other Bone** field, drag the root bone of the region you want (e.g. the `Thigh_L` bone for the left leg)
2. Enable **Include Child Bones** to capture the full leg (shin, foot, toes)
3. Click **Keep ONLY Other Part** or **Remove Other Part** depending on what you need
4. Save the result

Copyright Chris Burns

3.3.4 LOD Mesh Generation

Reduce draw call complexity by creating a simplified lower-body or upper-body mesh for distant LOD levels.

1. Extract the lower body using the **Other Bone** field (assign `Hips` or `Spine` as the root)
2. After saving, import the mesh back and apply a Unity **Mesh Simplifier** or similar to reduce poly count
3. Assign the simplified mesh as a lower LOD level in a `LODGroup` component

3.3.5 Gameplay Dismemberment Setup

Pre-separate body parts for a damage system where arms, legs, or the head can detach.

1. Extract each body region as a separate mesh asset (Head, Arms, LeftLeg, RightLeg, Torso)
2. At runtime, disable the original `SkinnedMeshRenderer` on hit
3. Spawn the appropriate pre-extracted mesh as a `MeshRenderer` (no longer needing to be skinned) with physics applied

> **Editor-only tool**
>
> Skinned Mesh Separator runs in the Unity Editor only. The mesh splitting happens at author time — the extracted `.asset` files are what you use at runtime. See Limitations for details.

Copyright Chris Burns

4. Reference

4.1 All Features

A complete reference of everything included in Skinned Mesh Separator.

4.1.1 Editor Window

Accessible via **Tools** → **Skinned Mesh Separator**. Dockable anywhere in the Unity editor layout.

4.1.2 Bone Assignment

Field	Purpose
Skinned Mesh Renderer	The source character mesh to extract from
Head Bone	Root bone for the head region
Left Shoulder Bone	Root bone for the left arm
Right Shoulder Bone	Root bone for the right arm
Other Bone	Any additional custom bone region
Include Child Bones	Whether to include all child bones in extraction

4.1.3 Auto-Find Bones

Scans the rig's full bone array and matches common naming patterns for head and shoulder bones. Supports:

- Whitespace, underscore, and hyphen separators
- Case-insensitive matching
- Common conventions: `Bip01`, `mixamo`, standard humanoid, and more

Console output confirms which bones were found and which were not.

4.1.4 One-Click Read/Write Toggle

Enables `isReadable` on the source mesh's `ModelImporter` and triggers an asset reimport — all from within the tool window. No need to navigate to the FBX import settings manually.

Copyright Chris Burns

4.1.5 Preset Extraction Buttons

Six preset buttons covering the most common workflows:

- Keep ONLY Head
- Remove Head (Keep Body & Arms)
- Keep ONLY Arms
- Remove Arms (Keep Body & Head)
- Keep ONLY Other Part
- Remove Other Part

4.1.6 Mesh Processing

- Reads `BoneWeight` data per vertex to identify influenced vertices
- Filters triangles per submesh — preserves submesh structure
- Outputs a new `Mesh` asset via `AssetDatabase.CreateAsset`
- Logs removed triangle count to the Console

4.1.7 Supported Mesh Types

- Any `SkinnedMeshRenderer` with Read/Write enabled
- Humanoid, Generic, and Custom rigs
- Meshes imported from FBX, OBJ, Blender, Maya, 3ds Max, and any standard format
- Any vertex count
- Multi-submesh meshes

4.1.8 Package Contents

- `SkinnedMeshSeparator.cs` — full C# source, commented
- Documentation / usage guide
- All source code is included and editable

Copyright Chris Burns

4.2 Limitations

Understanding these limitations will help you decide whether Skinned Mesh Separator fits your use case — and how to work around them when needed.

4.2.1 Open Geometry at Cut Boundaries

What happens: When a region is removed from a mesh, the boundary edges where triangles were deleted are left open (non-watertight). The tool does not automatically cap or close these holes.

Why: Closing open mesh geometry requires predicting the intended shape at the cut, which is highly topology-dependent and cannot be reliably automated for arbitrary character meshes.

When this matters:

- Real-time rendering of the cut edge at close range (visible seams)
- Physics simulations that require a closed mesh collider

When this does not matter (most use cases):

- The extracted mesh is hidden under clothing, armour, or another mesh
- You are masking or toggling the region at runtime (clothing swap systems)
- The extracted mesh is used as a `MeshRenderer` prop (dismemberment, LODs)
- The open edge faces away from the camera

> **Workaround**
>
> For cases where closed geometry is required, the extracted mesh can be brought into Blender or another DCC tool to fill the open edge loop manually. The time saving from automated extraction still applies — you only need to close the boundary, not re-split the mesh.

4.2.2 Editor-Only

Skinned Mesh Separator is a Unity Editor tool. It cannot be used at runtime (during Play mode or in a build).

The intended workflow is:

1. Extract meshes in the Editor using the tool
2. Save the extracted `.asset` files to your project
3. Use those mesh assets at runtime via `SkinnedMeshRenderer` components or `MeshRenderer` props

If you need to split meshes procedurally at runtime (e.g. dynamically dismember any character, not just pre-prepared ones), this tool is not the right solution. A runtime mesh splitting library would be required for that use case.

4.2.3 Read/Write Requirement

The source mesh must have **Read/Write Enabled** in its import settings. This is a Unity API requirement for accessing vertex data at edit time, not a limitation of this tool. The tool provides a one-click button to enable this setting.

Copyright Chris Burns

4.2.4 Bone Weight Threshold

Extraction is based on whether a vertex has any weight assigned to the target bone (including very small weights). There is no minimum weight threshold. In practice this is rarely an issue, but on meshes with heavy weight painting bleed between regions, a small number of boundary triangles may be captured in or excluded from the extraction unexpectedly.

Copyright Chris Burns

5. Support

5.1 Discord

The fastest way to get help. Join the Cerberus Bytes support server and ask in the relevant channel.

Join Discord

5.2 Email

For detailed bug reports or private enquiries:

info@cerberusbytes.com

Please include:

- Your Unity version
- The rig type (humanoid / generic / custom)
- A description of the issue and steps to reproduce
- Any relevant Console error messages

5.3 Asset Store

For refund requests or store-related issues, use Unity's Asset Store support system.

Asset Store Page

5.4 Leaving a Review

Reviews and ratings help other developers discover the asset and fund continued development. If Skinned Mesh Separator saved you time, a quick rating on the Asset Store is genuinely appreciated.

Leave a Rating

You can also click the ★ **Leave a Rating** button at the bottom of the tool window directly from within Unity.

5.5 Changelog

5.5.1 Version 1.1

- Added 🔍 **Auto-Find Bones** button — automatically detects head and shoulder bones from the rig
- Added footer buttons: Leave a Rating, Discord, Website
- Minor UI improvements

Copyright Chris Burns

5.5.2 Version 1.0

- Initial release
- Bone-based mesh extraction (head, arms, other)
- Preset buttons for common body regions
- Inverse selection (remove / keep)
- Child bone inclusion toggle
- Read/Write auto-enable button

5.6 Other Assets by Chris Burns

Browse the full catalogue on the Unity Asset Store:

View All Assets

Copyright Chris Burns